Script 01 — Generate Dataset: Notas explicativas

> Documento de seguimiento del script `scripts/01_generate_dataset.py`. Recoge las decisiones de diseño y la lógica detrás de cada bloque clave.

Propósito del script

Genera el dataset sintético inicial de e-commerce para el proyecto. Crea ~5.000 pedidos repartidos entre ~1.500 clientes únicos, con datos coherentes por locale (UK, IE, DE, FR, ES, NL).

Output: `data/raw/customers_orders.csv`

La columna `customer_notes` se deja **vacía intencionalmente** — la rellena Ollama en el siguiente script.

Decisiones clave del diseño

1. `RANDOM_SEED = 42`

Asegura que el dataset sea **reproducible**. Si alguien clona el repositorio y ejecuta el script, obtiene exactamente las mismas filas que tú.

Es estándar profesional para que los resultados sean replicables. Sin seed, cada ejecución produce datos distintos y el análisis no se puede comparar entre sesiones.

> Nota: el número 42 es convención (referencia a "Hitchhiker's Guide to the Galaxy"). Cualquier número entero sirve.

---

2. `fakers = {locale: Faker(locale) for locale in LOCALES}`

Truco importante. En lugar de mezclar locales en una sola instancia (lo que daría direcciones franco-españolas raras tipo "Calle Champs-Elysées 12, Madrid"), se crea una **instancia por país**.

Cuando se genera un cliente francés, se usa el Faker francés para todo:

- nombre francés
- dirección francesa
- teléfono con formato francés
- código postal francés

Todo coherente entre sí. El cliente parece real, no un Frankenstein de varios países.

Patrón a recordar: cuando trabajes con datos multi-locale, mantén los generadores separados y elige cuál usar según la "identidad" de la entidad.

---

3. Sistema de pesos con `random.expovariate`

Modela la **ley 80/20 del e-commerce real**: pocos clientes hacen muchos pedidos (los leales), muchos clientes hacen 1-2 pedidos (los ocasionales).

Sin este sistema, los pedidos se repartirían uniformemente entre clientes y el dataset perdería todo realismo: cada cliente tendría aproximadamente el mismo número de pedidos, lo cual no ocurre nunca en datos reales.

Resultado esperado: el top 5 de clientes con más pedidos debería tener entre 15-40 pedidos cada uno, mientras la mediana es de 1-3 pedidos.

> Por qué importa: muchos análisis de e-commerce se basan en identificar a los clientes leales (CLV — Customer Lifetime Value). Sin la distribución 80/20, esos análisis no tendrían sentido sobre el dataset.

---

4. Valores de pedido escalados por segmento

Los pedidos no tienen valores aleatorios uniformes — se escalan según el segmento del cliente:

- **VIP** → 80€ a 500€ por pedido
- **Regular** → 25€ a 200€ por pedido
- **New** → 10€ a 80€ por pedido

Esto crea **patrones reales** en los datos que luego harán que el análisis sea interesante.

El insight clave de privacidad: también permite demostrar k-anonymity de verdad. Si un solo cliente VIP gasta 500€ en una categoría rara desde un código postal específico, es **re-identificable** aunque le quites el nombre. La combinación única de (segmento + categoría + ubicación + valor alto) lo señala.

> Vas a poder mostrar exactamente este escenario en el proyecto: una query que detecta combinaciones de atributos que aparecen una sola vez en el dataset, lo cual viola k-anonymity.

---

5. `customer_notes` queda vacío

Decisión intencional. Lo rellena Ollama en el siguiente script (`02_generate_notes.py`) usando los datos de cada pedido como contexto para generar notas realistas.

Por qué separar los dos pasos:

- Buena ingeniería: si Ollama falla a mitad de ejecución (timeout, error de conexión, modelo descargado), el dataset estructurado está a salvo y no hay que regenerar todo.

- Velocidad: la generación estructurada es instantánea (segundos). La generación con Ollama es lenta (minutos a horas según el volumen).
- Iteración: puedes ajustar el prompt de Ollama varias veces sin tocar las 19 columnas estructuradas.
- Coste mental: un error en la lógica de Ollama no contamina los datos transaccionales.

Patrón a recordar: cuando un pipeline tiene un paso "rápido y determinista" y otro "lento y estocástico", sepáralos en scripts distintos. Es más robusto y más debuggeable.

Próximo paso

`02_generate_notes.py` → llamar a Ollama por cada pedido con un prompt que use los datos del cliente como contexto, generar la nota de servicio al cliente, y guardar el dataset enriquecido.

Esa nota es la que contendrá PII embebido en texto libre y justifica todo el bloque "Ollama" del proyecto.

Validación post-ejecución

Después de correr el script, verificar:

1. ✅ El CSV se generó en `data/raw/customers_orders.csv`.
2. ✅ Las primeras filas se ven coherentes (cliente alemán → nombre y dirección alemanes).
3. ✅ El top 5 de clientes con más pedidos tiene 15-40 pedidos (distribución 80/20 funcionando).
4. ✅ Los 6 países están representados sin desequilibrio extremo.
5. ✅ Los valores de pedido siguen el escalado por segmento (VIP gasta más en promedio).